

Flower Classification

<https://github.com/aiyshawary/flower-classification-project>

Python

TensorFlow

Keras

OpenCV

scikit-learn

Matplotlib

Seaborn

OVERVIEW

A CNN-based image classification project that identifies five flower types (Daisy, Dandelion, Rose, Sunflower, Tulip) using the Kaggle Flowers Recognition dataset and a custom TensorFlow/Keras model pipeline.

IMPACT

Built an end-to-end notebook workflow that prepares the dataset, trains a custom CNN, and evaluates multi-class accuracy to classify flower species from images.

TECHNICAL WALKTHROUGH

Dataset & Setup

Used the Kaggle Flowers Recognition dataset with five classes: Daisy, Dandelion, Rose, Sunflower, and Tulip. The dataset is organized into class folders and loaded into the notebook for training and validation.

1. Install dependencies: TensorFlow, OpenCV, scikit-learn, matplotlib, seaborn.
2. Place the dataset under `flowers/<class_name>/` to match the expected directory structure.

Preprocessing Pipeline

Images are resized to a consistent shape, normalized to [0,1], and split into train/validation sets to ensure fair evaluation across classes.

1. Resize and normalize images to stabilize training.
2. Create stratified train/validation splits for balanced class coverage.

Model Architecture

Implemented a custom CNN in Keras with stacked convolution + pooling blocks followed by dense layers for multi-class classification.

1. Convolutional blocks learn spatial features of petals and textures.
2. Dense layers map extracted features to 5-class softmax outputs.

Training & Evaluation

Tracked training and validation curves to monitor convergence and prevent overfitting.

1. Visualized accuracy/loss trends to validate model stability.

2. Evaluated class-wise performance using predictions on validation data.

Reproducible Notebook

The full workflow is implemented in the notebook for end-to-end reproducibility and easy experimentation.

KEY DETAILS

1. Organized the Kaggle Flowers Recognition dataset into class-specific folders for five flower categories.
2. Preprocessed images by resizing and normalizing pixel values, then created train/validation splits for model evaluation.
3. Implemented a custom CNN architecture in TensorFlow/Keras for 5-class image classification.
4. Tracked training/validation accuracy and loss across epochs and visualized results with Matplotlib/Seaborn.
5. Packaged the workflow into a reproducible Jupyter Notebook for easy experimentation and iteration.